

BIG DATA ANALYTICS

UNIT – III: SQOOP, NoSQL & HBase

Question Bank with Detailed Answers

PART – A: Short Answer Questions (2–5 Marks Each)

The following 10 short questions cover all major topics in Unit III.

Q1. What is SQOOP? Expand the acronym and explain its purpose.

Answer:

SQOOP stands for SQL + HADOOP. The name is derived from the first two letters of SQL and the last three letters of HADOOP, forming SQOOP.

SQOOP is a command-line tool used to transfer bulk data between Relational Database Management Systems (RDBMS) and Hadoop Distributed File System (HDFS). It was developed to solve the problems administrators faced when working with Hadoop:

- Maintaining data consistency between RDBMS and Hadoop
- Ensuring efficient utilization of resources
- Loading bulk data to Hadoop was not previously possible
- Loading data using manual scripts was very slow

SQOOP supports two primary operations: Import (RDBMS → HDFS) and Export (HDFS → RDBMS). It uses the YARN framework internally and leverages MapReduce for parallel data transfer with fault tolerance.

Q2. List and briefly explain any four features of SQOOP.

Answer:

SQOOP offers the following key features:

- 1. Parallel Import/Export:** SQOOP uses the YARN framework to import and export data. This provides fault tolerance on top of parallelism — multiple map tasks run simultaneously, each handling a portion of data.
- 2. Full Load:** SQOOP can load the entire table or all tables from a database in a single command, making it efficient for complete data migrations.
- 3. Incremental Load:** SQOOP can load only the newly added or recently modified rows from a table. This is useful for regular synchronization jobs where only new data needs to be transferred.
- 4. Load Data Directly into Hive/HBase:** SQOOP can directly import data into Apache Hive data warehouse or HBase (a NoSQL database), enabling immediate analytical querying without intermediate steps.
- 5. Compression:** Data can be compressed using the deflate (gzip) algorithm with the `--compress` argument, reducing storage requirements.

Q3. Explain the SQOOP Import command with syntax and example.

Answer:

The SQOOP Import tool is used to import individual tables or entire databases from RDBMS into HDFS, Hive, or HBase.

Syntax:

```
$ sqoop import (generic-args) (import-args)
```

Example: To import the 'emp' table from MySQL database 'userdb':

```
$ sqoop import \  
--connect jdbc:mysql://localhost/userdb \  
--username root \  
--table emp --m 1
```

Here:

- --connect specifies the JDBC connection URL
- --username provides the database username
- --table specifies the table to import
- --m 1 specifies the number of map tasks

After import, to verify: `$ $HADOOP_HOME/bin/hadoop fs -cat /emp/part-m-*`

The data appears in HDFS as comma-separated values: 1201, gopal, manager, 50000, TP

Q4. What is SQOOP Export? Write its syntax and working.

Answer:

SQOOP Export is the reverse of SQOOP Import. It exports a set of files from HDFS back to an RDBMS (like MySQL, Oracle, SQL Server).

Syntax:

```
$ sqoop export (generic-args) (export-args)
```

Working of SQOOP Export:

- The client submits the export command
- SQOOP first examines the database for metadata (schema/column info)
- It then submits a Map-Only job to YARN
- Each map task reads a chunk of HDFS data and writes it to the RDBMS table using JDBC
- The whole dataset is reconstructed at the destination by combining all exported chunks

Example: Export data from /emp/emp_data in HDFS to 'employee' table in MySQL:

```
$ sqoop export \  
--connect jdbc:mysql://localhost/db \  
--username root \  
--table employee \  
--export-dir /emp/emp_data
```

Q5. What is Incremental Import in SQOOP? Name its two modes.

Answer:

Incremental Import is a technique that imports only newly added or recently modified rows from a table, rather than importing the entire table again. It is used for efficient, periodic synchronization.

Three arguments are required:

- `--check-column`: Specifies the column to examine when determining new rows (cannot be CHAR/VARCHAR type)
- `--incremental`: Specifies the mode (append or lastmodified)
- `--last-value`: The maximum value of the check column from the previous import

The two modes of Incremental Import are:

1. --incremental append: Fetches only newly added rows (inserted rows). It does NOT fetch updated rows. Uses the primary key column as check-column.

2. --incremental lastmodified: Fetches only rows that have been modified or updated since the last import. It does NOT import newly added rows. Uses a timestamp column to detect changes.

Q6. What is NoSQL? What are the types of NoSQL databases?

Answer:

NoSQL stands for 'Not Only SQL'. It is a type of Database Management System (DBMS) designed to handle and store large volumes of unstructured and semi-structured data. Unlike RDBMS, NoSQL:

- Does not require a fixed schema
- Avoids complex joins
- Is easy to scale horizontally
- Handles Big Data and real-time web applications

Companies like Twitter, Facebook, and Google use NoSQL databases to manage terabytes of data daily.

Types of NoSQL Databases:

1. Key-Value Pair: Stores data as a hash table of unique key-value pairs. Examples: Redis, Dynamo, Riak.

2. Column-Oriented: Stores data as column families. Optimized for aggregations (SUM, COUNT, AVG). Examples: HBase, Cassandra.

3. Document-Oriented: Stores data as JSON or XML documents. Examples: MongoDB, CouchDB.

4. Graph-Based: Stores data as nodes and edges for complex relationships. Examples: Neo4J, OrientDB.

Q7. What is HBase? How is it different from traditional HDFS?

Answer:

HBase is a distributed, column-oriented NoSQL database built on top of the Hadoop File System (HDFS). It is modeled after Google's BigTable and is an open-source, horizontally scalable project.

HBase provides random, real-time read/write access to data stored in HDFS. While HDFS only supports sequential (batch) access, HBase enables random access to any data point at any time.

Key differences:

HDFS	HBase
Only batch/sequential processing	Random real-time read/write access
Not suitable for small updates	Supports row-level updates

No direct querying of records	Supports querying via row key
-------------------------------	-------------------------------

Q8. Explain the HBase Architecture with its main components.

Answer:

HBase architecture consists of four main components:

- 1. HMaster:** The Master server that assigns regions to Region Servers, performs DDL operations (CREATE/DELETE tables), monitors all Region Server instances, and controls load balancing and failover.
- 2. Region Server:** Region Servers handle data read and write requests from clients. They contain Regions (horizontal partitions of a table), each of default size 256 MB. Internally each region contains a Store, which has a MemStore (cache memory for writes) and HFiles (persistent storage on HDFS).
- 3. ZooKeeper:** Acts as a coordinator for the cluster. It maintains configuration information, provides distributed synchronization, detects server failures, and enables clients to communicate with Region Servers.
- 4. HDFS:** The underlying distributed file system where all actual data is stored persistently. Region Servers run on HDFS DataNodes.

Flow: Client → ZooKeeper → Region Server → HDFS. The ZooKeeper helps the client locate the appropriate Region Server, which then reads/writes from/to HDFS.

Q9. What are CRUD operations in HBase? Give syntax for each.

Answer:

CRUD stands for Create, Read, Update, and Delete — the four fundamental database operations.

C – CREATE (INSERT): Add or insert data into the table.

```
CREATE TABLE Table_Name (ColumnName1 Datatype, ColumnName2 Datatype, ...);
```

R – READ (SELECT): Retrieve or fetch data from the table.

```
SELECT * FROM TableName;           -- All records
SELECT * FROM TableName WHERE CONDITION;  -- Conditional
```

U – UPDATE: Modify existing records in the table.

```
UPDATE Table_Name SET ColumnName = Value WHERE CONDITION;
```

D – DELETE: Remove records from the table.

```
DELETE FROM TableName;           -- All records
DELETE FROM TableName WHERE CONDITION;  -- Specific records
```

In HBase shell, these translate to put (insert/update), get (read single row), scan (read multiple rows), and delete commands.

Q10. What is the storage mechanism in HBase? Explain with table structure.

Answer:

HBase is a column-oriented database where tables are sorted by row. The storage hierarchy is:

- Table → collection of Rows
- Row → collection of Column Families
- Column Family → collection of Columns

- Column → collection of Key-Value pairs

The table schema defines only Column Families (not individual columns). Each column family can have any number of columns, and columns can vary per row. Each cell has a timestamp for versioning.

Example of Column Families in an employee HBase table:

Row Key	Column Family: Personal Data		Column Family: Professional Data	
empid	name	city	designation	salary
1	raju	hyderabad	manager	50,000
2	ravi	chennai	sr.engineer	30,000
3	rajesh	delhi	jr.engineer	25,000

PART – B: Long Answer Questions (10–15 Marks Each)

The following 5 long questions require detailed explanations with examples.

QL1. Explain SQOOP in detail — its Introduction, Architecture, Features, Advantages, and Disadvantages. Also explain SQOOP Import with working and examples.

Detailed Answer:

1. Introduction to SQOOP

SQOOP (SQL + HADOOP) is a command-line tool used to transfer bulk data between Relational Database Management Systems (RDBMS) and Hadoop. Before SQOOP, administrators faced several challenges:

- Maintaining data consistency when moving data between RDBMS and Hadoop
- Ensuring efficient resource utilization during data transfers
- Loading bulk data to Hadoop was not possible through standard means
- Manual scripts for data loading were slow and error-prone

SQOOP solved all these problems by providing a standardized, parallel, fault-tolerant data transfer mechanism. It supports two core operations: Import (RDBMS → Hadoop) and Export (Hadoop → RDBMS).

2. SQOOP Architecture

The SQOOP architecture involves the following flow:

- Client submits import/export command to SQOOP
- SQOOP has connectors for Enterprise Data Warehouses, Document-Based Systems, and Relational Databases
- SQOOP introspects the database to gather metadata (primary key information)
- It submits a Map-Only job to the Hadoop YARN cluster
- Multiple Map tasks run in parallel, each handling a split of data
- For Import: Map tasks read from RDBMS and write to HDFS/Hive/HBase
- For Export: Map tasks read from HDFS and write to RDBMS via JDBC

3. SQOOP Features

Parallel Import/Export: Uses YARN framework. Multiple mappers run in parallel, providing both speed and fault tolerance.

Import SQL Query Results: Can import the result of any custom SQL query directly into HDFS.

Connectors for All Major RDBMS: Supports MySQL, Oracle, Microsoft SQL Server, DB2, PostgreSQL, and more.

Kerberos Security Integration: Supports Kerberos authentication protocol for secure communication over insecure networks.

Full and Incremental Load: Full load imports entire tables; Incremental load imports only new or updated rows.

Compression: Supports gzip compression using `--compress` or `--compression-codec` arguments.

4. SQOOP Import — Detailed Working

The SQOOP Import tool imports data from RDBMS tables into HDFS (or Hive/HBase).

Syntax: `$ sqoop import (generic-args) (import-args)`

Working Steps:

1. The import command is submitted by the user with JDBC connection parameters
2. SQOOP introspects the database to get metadata (column types, primary keys)
3. It divides the input dataset into splits based on the number of mappers (--m flag)
4. Each mapper connects to the database using JDBC and fetches its assigned data partition
5. Mappers write the data to HDFS as text files (or binary Sequence files)

Example — Import the 'emp' table from MySQL:

```
$ sqoop import \  
  --connect jdbc:mysql://localhost/userdb \  
  --username root \  
  --table emp --m 1
```

Verify imported data:

```
| $ $HADOOP_HOME/bin/hadoop fs -cat /emp/part-m-*
```

Output: 1201, gopal, manager, 50000, TP

Import with target directory:

```
$ sqoop import --connect jdbc:mysql://localhost/userdb \  
  --username root --table emp_add --m 1 --target-dir /queryresult
```

Import with WHERE condition (subset of table):

```
$ sqoop import --connect jdbc:mysql://localhost/userdb \  
  --username root --table emp_add --m 1 \  
  --where "city='sec-bad'" --target-dir /wherequery
```

5. Advantages of SQOOP

- Supports data transfer with a variety of structured data stores (Oracle, MySQL, etc.)
- Performs ETL (Extract, Transform, Load) operations efficiently
- Parallel processing of data accelerates the overall transfer
- Uses MapReduce mechanism, providing inherent fault tolerance

6. Disadvantages of SQOOP

- Failure during operations requires special handling and recovery
- Uses JDBC connections which are relatively slow and inefficient
- Export performance depends on the hardware configuration of the RDBMS

QL2. Explain SQOOP Export and Incremental Import in detail with syntax, working, and examples for both append and lastmodified modes.

Detailed Answer:

1. SQOOP Export

SQOOP Export transfers data from HDFS (or Hive/HBase) back to an RDBMS. It is the reverse of SQOOP Import and is typically done after processing data in Hadoop and writing results back to operational databases.

Syntax: \$ sqoop export (generic-args) (export-args)

Working Steps:

6. The client submits the export command to SQOOP
7. SQOOP examines the RDBMS database for metadata (column names, data types)

8. It submits a Map-Only job to YARN
9. Each mapper reads a chunk of HDFS data and converts rows into RDBMS records
10. Mappers write chunks to the RDBMS using JDBC connections
11. The complete dataset is assembled at the destination by combining all mapper outputs

Example:

```
$ sqoop export \  
  --connect jdbc:mysql://localhost/db \  
  --username root \  
  --table employee \  
  --export-dir /emp/emp_data
```

Verification in MySQL:

```
| mysql> select * from employee;
```

This would display all employee records that were previously stored in HDFS.

2. Incremental Import

Incremental Import is an optimization technique that avoids re-importing entire tables. Instead, it selectively imports only:

- Newly appended rows (using append mode)
- Recently modified rows (using lastmodified mode)

Required Arguments:

Argument	Description
--check-column	Column to examine for new rows (cannot be CHAR/VARCHAR type)
--incremental	Mode of incremental import: 'append' or 'lastmodified'
--last-value	Maximum value of check column from the previous import run

Mode 1: --incremental append

This mode imports only newly INSERT-ed (appended) rows since the last import. It does NOT pick up rows that were updated.

Scenario: A new employee (id 1206, satish p) was added to the emp table after the last import.

```
$ sqoop import \  
  --connect jdbc:mysql://localhost/userdb \  
  --username root \  
  --table emp --m 1 \  
  --incremental append \  
  --check-column id \  
  --last-value 1205
```

SQOOP will only import rows where id > 1205, i.e., only the new row: 1206, satish p, grp des, 20000, GR

Mode 2: --incremental lastmodified

This mode imports only rows that have been modified (updated) after a given timestamp. It does NOT import new rows.

Scenario: Employee id 1203 (Kalil) had salary updated from 30000 to 60000.


```
$ sqoop import \  
--connect jdbc:mysql://localhost:3306/db \  
--username root -P \  
--table emp \  
--target-dir /user/hive/warehouse/emp \  
--append \  
--check-column last_updated_timestamp \  
--incremental lastmodified \  
--last-value "2024-01-01 00:00:00"
```

Only the modified row for Kalil (id 1203, salary 60000) will be imported.

QL3. What is NoSQL? Explain its features, types with examples, and compare advantages/disadvantages with RDBMS.

Detailed Answer:

1. Introduction to NoSQL

NoSQL (Not Only SQL) is a category of database management systems designed to handle large volumes of unstructured, semi-structured, or structured data that traditional RDBMS struggle to manage efficiently.

NoSQL databases became popular because of the limitations of RDBMS at internet scale. Companies like Google, Facebook, Amazon, Twitter handle terabytes of data every day — and RDBMS systems become too slow and expensive to maintain at that scale.

Two scaling approaches exist:

Scale Up (Vertical): Adding more RAM, CPU, or HDD to an existing machine. This is expensive and has physical limits.

Scale Out (Horizontal): Distributing the database across multiple commodity machines. NoSQL specializes in this approach — databases like MongoDB and Cassandra can scale out by simply adding more nodes.

2. Features of NoSQL

Non-Relational: NoSQL databases do not follow the relational model. They do not provide tables with fixed-column records and do not require object-relational mapping or data normalization.

Schema-Free: NoSQL databases are either schema-free or have relaxed schemas. No schema definition is required before storing data. Different rows can have different structures (heterogeneous data).

Simple API: Offers easy-to-use interfaces for storage and querying. APIs allow low-level data manipulation. Text-based protocols like HTTP REST with JSON are commonly used.

Distributed: Multiple NoSQL instances can run in a distributed fashion. Provides auto-scaling and failover. Uses asynchronous multi-master replication or peer-to-peer replication.

Horizontal Scalability: Adding more nodes to the cluster seamlessly increases capacity and throughput.

3. Types of NoSQL Databases

A) Key-Value Pair Based: Data is stored as a hash table where each entry has a unique key and an associated value (string, JSON, list, etc.). This is the simplest NoSQL model.

Example: { "Website": "GeeksforGeeks" } or { "user:1001": { "name": "Alice", "age": 25 } }

Best for: Session management, shopping carts, caching.

Examples: Redis, Amazon DynamoDB, Riak

B) Column-Oriented (Column-Family): Data is organized by columns (column families) rather than rows. Extremely efficient for analytical queries on specific columns across millions of rows.

Best for: Data warehouses, Business Intelligence, CRM, time-series data.

Examples: HBase, Apache Cassandra

C) Document-Oriented: Stores data as documents (JSON or XML format). Each document is self-describing and can have a variable structure. No joins needed — all related data is in one document.

Example: { "_id": 101, "name": "Alice", "orders": [{"item": "book", "qty": 2}] }

Best for: Blogging platforms, e-commerce, real-time analytics.

Examples: MongoDB, CouchDB, Amazon SimpleDB

D) Graph-Based: Stores data as nodes (entities) and edges (relationships). Ideal for highly connected data. Traversals are extremely fast because relationships are pre-computed.

Example: A Person node connected to a City node via LivesIn edge, connected to a Restaurant via Likes edge.

Best for: Social networks, logistics, fraud detection, recommendation engines.

Examples: Neo4J, InfiniteGraph, OrientDB

4. Advantages of NoSQL

- High Scalability: Horizontal sharding distributes data across machines for virtually unlimited scale
- Flexibility: Handles dynamic, evolving data models without schema migrations
- High Availability: Auto-replication ensures data is always accessible even during node failure
- Performance: Optimized for large-scale, high-traffic scenarios
- Cost-Effectiveness: Runs on commodity hardware; less complex than RDBMS

5. Disadvantages of NoSQL

- Lack of Standardization: Each NoSQL system has its own query language and APIs
- Lack of ACID Compliance: Strong consistency guarantees are not always provided
- Complex Management: Managing distributed NoSQL clusters is more complex than RDBMS
- Weak Query Support: Not designed for complex multi-table analytical queries
- Backup Issues: Some NoSQL databases (like older MongoDB versions) lack consistent backup mechanisms

QL4. Explain HBase Architecture in detail — HMaster, Region Server, ZooKeeper, and HDFS. Include the internal structure of Region Server (MemStore, HFile, Store).

Detailed Answer:

Overview of HBase

HBase is a distributed, column-oriented NoSQL database built on top of HDFS. It is designed to provide random, real-time read/write access to big data, solving Hadoop's limitation of only supporting sequential batch processing.

HBase Architecture consists of 4 major components: HMaster, Region Server, ZooKeeper, and HDFS.

1. HMaster

HMaster is the master server in HBase. It performs the following functions:

- Assigns Regions to Region Servers when the cluster starts or when a Region Server fails
- Handles DDL operations — creating tables, deleting tables, modifying column families
- Monitors all Region Server instances in the cluster through ZooKeeper heartbeats
- Controls load balancing — redistributes regions when some servers are overloaded
- Handles failover — reassigns regions from failed servers to healthy ones
- In a distributed environment, HMaster runs several background threads for these tasks

Note: Clients do NOT communicate with HMaster for actual data reads/writes. HMaster only handles metadata and administrative operations.

2. Regions and Region Servers

Regions: A Region is the basic unit of scalability in HBase. HBase tables are divided horizontally by row key range into Regions. Each region is a subset of the table's rows. The default maximum size of a region is 256 MB. When a region grows beyond this, it is automatically split into two smaller regions.

Region Server: A Region Server is a process that manages one or more regions. It handles all client read and write requests for its assigned regions. Region Servers run on HDFS DataNodes in the Hadoop cluster.

Internal structure of a Region Server:

- Each Region Server contains multiple Regions
- Each Region contains one or more Stores (one per column family)
- Each Store contains one MemStore (write cache) and zero or more HFiles (persistent storage)

Internal Structure: MemStore and HFile

MemStore (Write Cache): When data is written to HBase, it is first stored in the MemStore, which is an in-memory write buffer. Once the MemStore fills up (default threshold: 128 MB), it is flushed to disk as an HFile. This design makes writes very fast.

HFile (Store File): HFiles are the persistent storage format on HDFS. They are sorted, immutable files. When multiple HFiles accumulate, they are periodically merged through a compaction process to improve read performance.

Write-Ahead Log (WAL): Before data is written to the MemStore, it is recorded in the Write-Ahead Log (WAL/HLog) on HDFS. This ensures data durability — if a Region Server crashes before the MemStore is flushed, the data can be recovered from the WAL.

3. ZooKeeper

ZooKeeper acts as the coordinator and registry for the entire HBase cluster. Its responsibilities include:

- Maintaining configuration information for the cluster
- Providing distributed synchronization between HBase components
- Detecting server failures via heartbeat monitoring
- Notifying the HMaster of Region Server failures
- Storing the location of the META table (which maps row keys to regions)
- All clients first contact ZooKeeper to find the Region Server responsible for their data

ZooKeeper runs as a distributed ensemble (usually 3 or 5 nodes) to avoid single points of failure.

4. HDFS

HDFS (Hadoop Distributed File System) is the underlying storage layer for HBase. All data written to HBase is ultimately persisted in HDFS:

- HFiles are stored in HDFS
- Write-Ahead Logs (WAL) are stored in HDFS

- HDFS provides fault tolerance through block replication (typically 3 replicas)
- Region Servers run co-located with HDFS DataNodes for data locality

Read and Write Path

Write Path: Client → ZooKeeper (find Region Server) → Region Server → WAL (HDFS) → MemStore → (flush) → HFile (HDFS)

Read Path: Client → ZooKeeper (find Region Server) → Region Server → Check MemStore → Check Block Cache → HFile (HDFS) → Return data

QL5. Explain the types of NoSQL databases in detail. Also explain the complete HBase CRUD operations with syntax, examples, and the HBase data model.

Detailed Answer:

PART I: Types of NoSQL Databases in Detail

1. Key-Value Pair Databases

Key-value stores are the simplest and most widely used type of NoSQL database. Data is stored as a dictionary (hash map) where each key is unique and maps to a value.

Data Model: { key → value } where value can be a string, integer, JSON object, list, or binary data.

Characteristics:

- Optimized for simple GET/SET/DELETE operations
- Extremely fast for high-throughput, low-latency workloads
- Values are opaque — the database does not interpret them
- Used as collections, dictionaries, and associative arrays

Real-World Example: Redis is used by Twitter to store user session tokens, and by gaming companies for leaderboards. Amazon DynamoDB powers Amazon's shopping cart.

Use Cases: Session management, caching, shopping carts, real-time leaderboards.

Popular Databases: Redis, Amazon DynamoDB, Riak, Memcached

2. Column-Oriented (Column-Family) Databases

Column-Family databases organize data by columns rather than rows. Related columns are grouped into Column Families, which are stored together on disk.

Data Model: Tables contain Row Keys, and each row can have multiple Column Families containing key-value columns.

Characteristics:

- Extremely efficient for read-heavy analytical workloads on specific columns
- High performance on aggregation queries: SUM, COUNT, AVG, MIN, MAX
- Sparse storage: rows can have different columns — columns without data take no space
- Designed for horizontal scaling to petabytes of data

Example (HBase table):

Row Key	personal_data:name	personal_data:city	prof_data:salary
1	raju	hyderabad	50,000
2	ravi	chennai	30,000

Use Cases: Data warehouses, time-series data, IoT sensor data, clickstream analytics.

Popular Databases: Apache HBase, Apache Cassandra, Google Bigtable

3. Document-Oriented Databases

Document-oriented databases store data as self-contained documents in JSON, BSON, or XML format. Each document can have a different structure.

Data Model: Collections of documents, where each document is a JSON/XML object.

Example document in MongoDB:

```
{ "_id": ObjectId("507f1f77"),  
  "name": "Alice",  
  "email": "alice@example.com",  
  "orders": [{ "item": "laptop", "qty": 1, "price": 50000 }]  
}
```

Characteristics:

- No schema enforcement — documents in the same collection can differ
- Supports nested data (arrays, embedded documents) avoiding joins
- Rich query language for filtering, aggregation, and text search
- Not suitable for complex multi-document transactions

Use Cases: Blogging platforms, e-commerce catalogs, content management, real-time analytics.

Popular Databases: MongoDB, CouchDB, Amazon SimpleDB, Riak

4. Graph-Based Databases

Graph databases are built to store and navigate complex, highly connected data. They represent data as nodes (entities) and edges (relationships).

Data Model: Nodes and Edges, each with properties (key-value pairs).

Example: In a social network:

- Node: Person (Alice), Person (Bob), City (Hyderabad), Company (TCS)
- Edge: Alice —[FRIENDS_WITH]→ Bob
- Edge: Alice —[LIVES_IN]→ Hyderabad
- Edge: Bob —[WORKS_AT]→ TCS

Characteristics:

- Traversing relationships is extremely fast (relationships are stored as direct pointers)
- Multi-relational in nature — unlike RDBMS where tables are loosely connected
- Every node and edge has a unique identifier
- Supports complex relationship queries in constant time regardless of dataset size

Use Cases: Social networks, recommendation engines, fraud detection, logistics routing.

Popular Databases: Neo4J, InfiniteGraph, OrientDB, Amazon Neptune

PART II: HBase CRUD Operations in Detail

HBase uses its own shell commands for CRUD operations. Below is each operation with syntax and examples.

C — CREATE

In HBase, creating a table requires specifying the table name and column families.

HBase Shell Syntax:

```
| create '<table_name>', '<column_family1>', '<column_family2>'
```

Example: Create an 'employee' table with two column families:

```
| hbase> create 'employee', 'personal_data', 'professional_data'
```

In SQL equivalent:

```
CREATE TABLE employee (  
    empid INT PRIMARY KEY,  
    personal_data VARCHAR(100),  
    professional_data VARCHAR(100)  
);
```

R — READ

Two main commands for reading: get (single row) and scan (multiple rows).

Get (single row):

```
hbase> get 'employee', '1'  
-- Returns all column families for row with key '1'
```

Scan (all rows):

```
hbase> scan 'employee'
```

Scan with filter:

```
hbase> scan 'employee', { COLUMNS => 'personal_data:name', LIMIT => 3 }
```

SQL equivalent:

```
SELECT * FROM employee;  
SELECT * FROM employee WHERE empid = 1;
```

U — UPDATE

In HBase, updates use the same 'put' command as inserts. Putting a value for an existing row key overwrites the previous value (the old version is retained with a timestamp for versioning).

```
hbase> put 'employee', '1', 'professional_data:salary', '60000'
```

This updates the salary of employee with row key '1' from its old value to 60000.

SQL equivalent:

```
UPDATE employee SET salary = 60000 WHERE empid = 1;
```

D — DELETE

Delete specific cell:

```
hbase> delete 'employee', '1', 'personal_data:city'
```

Delete entire row:

```
hbase> deleteall 'employee', '1'
```

Truncate entire table:

```
hbase> truncate 'employee'
```

Drop table (must disable first):

```
hbase> disable 'employee'  
hbase> drop 'employee'
```

SQL equivalent:

```
DELETE FROM employee WHERE empid = 1;  
DROP TABLE employee;
```

Summary of CRUD in HBase vs SQL

Operation	HBase Command	SQL Equivalent
Create (Insert)	put 'table', 'rowkey', 'cf:col', 'value'	INSERT INTO table VALUES (...)
Read (All Rows)	scan 'table'	SELECT * FROM table

Read (One Row)	get 'table', 'rowkey'	SELECT * WHERE id = rowkey
Update	put 'table', 'rowkey', 'cf:col', 'newval'	UPDATE table SET col = val WHERE id = k
Delete (Cell)	delete 'table', 'rowkey', 'cf:col'	DELETE col FROM table WHERE id = k
Delete (Row)	deleteall 'table', 'rowkey'	DELETE FROM table WHERE id = rowkey
Delete (Table)	disable 'table'; drop 'table'	DROP TABLE table